Verificações — Assignment #1

Verificações feitas em 12 de outubro de 2026 sobre o código do ramo `carolina`:

1. o BRIEF próprio comparado, afirmação a afirmação, com os dois artigos do BRIEF;
2. as funções e os parâmetros do OpenCV comparados com a versão instalada e com o enunciado.

Legenda: **Igual** (o código faz o que a fonte diz) · **Interpretação** (a fonte é ambígua e escolhemos uma leitura) · **Escolha nossa** (a fonte não diz nada) · **Diferente** (fazemos outra coisa, com motivo).

---

1. BRIEF próprio: artigo → código

Fontes: [ECCV] Calonder, Lepetit, Strecha, Fua, "BRIEF: Binary Robust Independent Elementary Features", ECCV 2010; [TPAMI] Calonder et al., "BRIEF: Computing a Local Binary Descriptor Very Fast", IEEE TPAMI 34(7), 2012. Código: `src/brief_own.py`, parâmetros em `CFG["brief_own"]`.

1.1 O descritor

#	O artigo diz	O código faz	Estado
1	[ECCV §3, Eq. 1] O teste τ vale 1 se p(x) < p(y) e 0 caso contrário	`bits = smooth[y + y1, x + x1] < smooth[y + y2, x + x2]`	Igual
2	[ECCV §3] "p(x) is the pixel intensity in a smoothed version of p"	As comparações são feitas na imagem suavizada (`smooth`), não na original	Igual
3	[ECCV §3, Eq. 2] O descritor é a cadeia de n_d bits dos testes	`np.packbits(bits, axis=1)` → `uint8 (N, n_d/8)`	Igual (a ordem dos bits dentro de cada byte é a do NumPy; não altera a distância de Hamming)
4	[ECCV §3] "we consider n_d = 128, 256, and 512"; BRIEF-k com k = n_d/8 bytes. [TPAMI §IV-A] "the default descriptor BRIEF-32"	`n_bits = 256` (BRIEF-32)	Igual
5	[ECCV §3] "Choosing a set of n_d (x, y)-location pairs uniquely defines a set of binary tests"	`make_pairs` gera os pares uma vez (seed fixa, com cache) e os mesmos pares são usados em todos os keypoints e nas duas imagens	Igual
6	[ECCV §1, §4] Os descritores comparam-se com a distância de Hamming	`cv2.NORM_HAMMING`	Igual

1.2 Suavização

#	O artigo diz	O código faz	Estado
7	[ECCV §3.1] Suavizar antes dos testes, porque testes em píxeis isolados são sensíveis ao ruído	`cv2.GaussianBlur` antes das comparações	Igual
8	[ECCV §3.1] "recognition rates remain relatively constant in the 1 to 3 range and, in practice, we use a value of 2"	`blur_sigma = 2.0`	Interpretação: o texto fala em "variances", mas as legendas das figuras (ECCV Fig. 1; TPAMI Fig. 1) identificam os valores testados como σ (0,65 … 3,05). Usamos σ = 2
9	[ECCV §3.1] "For the corresponding discrete kernel window we found a size of 9 × 9 pixels be necessary and sufficient"	`blur_ksize = 9`	Igual ao ECCV
10	[TPAMI §III-A] A mesma frase, mas com "7 × 7 pixels"	9 × 9	Diferente do TPAMI: seguimos a versão original; 9 é também o kernel do BRIEF do OpenCV
11	[TPAMI §III-A] Um filtro de caixa dá a mesma exatidão e é mais rápido (imagens integrais)	Gaussiana	Diferente: opção não usada. Consequência: o nosso descritor é mais lento (~15 ms contra ~4 ms do OpenCV, que usa filtro de caixa)

1.3 Disposição espacial dos testes

#	O artigo diz	O código faz	Estado
12	[ECCV §3.2] Patch de tamanho S × S, com a origem no centro do patch	Coordenadas dos pares relativas ao keypoint; `patch_size = 48`	Escolha nossa quanto ao valor: nenhuma das versões indica S. 48 é o valor do BRIEF do OpenCV (medimos a margem: 28 px = 48/2 + 9/2)
13	[ECCV §3.2] G I: (X, Y) ~ i.i.d. Uniform(−S/2, S/2)	`rng.uniform(-half, half, (n_bits, 2))` para X e para Y	Igual
14	[ECCV §3.2] G II: (X, Y) ~ i.i.d. Gaussian(0, $S^2/25$)	`rng.normal(0, S / 5, ...)` (σ = S/5 ⇔ $\sigma^2 = S^2/25$)	Igual
15	[ECCV §3.2] G III: X ~ Gaussian(0, $S^2/25$); Y ~ Gaussian(x_i, $S^2/100$)	X: `rng.normal(0, S / 5)`; Y: `rng.normal(x, S / 10)`	Igual
16	[ECCV §3.2] G III: "Test locations outside the patch are clamped to the edge of the patch"	`np.clip(..., -half, half)`	Igual. Interpretação num detalhe: encostamos X à borda antes de sortear Y em torno dele (o artigo não diz a ordem)
17	[ECCV §3.2] G IV: pares sorteados nas posições discretas de uma grelha polar grosseira	—	**Não implementada** (decisão: 4 das 5 estratégias; o enunciado pede pelo menos 2)
18	[ECCV §3.2] G V: x_i = (0, 0) e y_i "takes all possible values on a coarse polar grid containing n_d points"	`x = 0`; `y = polar_grid(S, n_bits, 8)` com 8 anéis × 32 ângulos = 256 pontos	Igual na definição; **escolha nossa** no nº de anéis e de ângulos (o artigo não os indica)
19	(não mencionado) Pontos fora do patch na G II	Encostados à borda, como na G III	Escolha nossa (com σ = S/5, ~1% das coordenadas cai fora)
20	(não mencionado) Passagem de coordenadas reais a píxeis	`np.rint` (píxel mais próximo)	Escolha nossa
21	(não mencionado) Keypoints junto à borda da imagem	Removidos a menos de 28 px da borda	Escolha nossa, igual ao comportamento do OpenCV

1.4 Propriedades e avaliação

#	O artigo diz	O código faz	Estado
22	[ECCV §4] "BRIEF is not designed to be rotationally invariant"; tolera 10 a 15 graus e depois cai	Sem orientação. No nosso teste: precisão 0,72–0,90 a 15° e ≤ 0,22 a 30°	Igual (comportamento reproduzido)
23	[ECCV §5] Invariância à orientação e à escala são trabalho futuro. [TPAMI §I, §IV-B, §IV-C] Define quatro variantes, {U, O, S, D}-BRIEF, conforme tenham ou não invariância à orientação e à escala	Sem orientação nem escala: corresponde ao BRIEF original do ECCV (a variante sem invariâncias do TPAMI)	Igual. As variantes com orientação/escala não foram implementadas
24	[ECCV §4] Usaram keypoints SURF e CenSurE; "any fast detector such as CenSurE or FAST can be used"	Keypoints FAST	Igual (o artigo indica o FAST como opção)
25	[ECCV §4] Matching por vizinho mais próximo (e, numa experiência, verificação esquerda-direita)	Ratio test 0,8 + cross-check	Diferente: usamos o protocolo do enunciado (§2.8), igual para todos os métodos
26	[ECCV §4] Medida: "recognition rate" com N keypoints correspondentes	PMR, precisão, matching score, recall	Diferente: usamos as métricas obrigatórias do enunciado
27	[ECCV §3.2, Fig. 3] A G V perde para todas as aleatórias; a G II tem pequena vantagem	G V com a precisão mais baixa em `i` e `v`; G II com o melhor matching score em ponto de vista; G III melhor em iluminação	Reproduzido em parte (o artigo testa a sequência Wall, de ponto de vista)

Resumo: 15 afirmações iguais, 2 interpretações (σ = 2; ordem do encosto na G III), 5 escolhas nossas onde o artigo é omisso (S, grelha polar, encosto na G II, arredondamento, margem), 4 diferenças justificadas (7×7 e filtro de caixa do TPAMI; matching e métricas do enunciado) e 1 estratégia não implementada (G IV).

---

2. OpenCV: funções e parâmetros

Versão instalada e verificada: `opencv-contrib-python 4.13.0.92` (`cv2.__version__` = 4.13.0). Os valores por omissão foram lidos dos próprios objetos (`getThreshold()` , etc.), não de memória.

2.1 Funções usadas e o que o enunciado indica (§11)

Função	Onde	Enunciado §11	Estado
`cv2.SIFT_create()`	`detectors.py`	"Use cv2.SIFT_create() for SIFT"	Igual
`cv2.ORB_create()`	`detectors.py`	"Use cv2.ORB_create() for ORB"	Igual
`cv2.KAZE_create()`	`detectors.py`	"Use cv2.KAZE_create() for KAZE"	Igual
`cv2.FastFeatureDetector_create()`	`detectors.py`	"Use cv2.FastFeatureDetector_create() for FAST"	Igual
`cv2.BFMatcher` (`knnMatch`)	`matching.py`	"Use cv2.BFMatcher or an appropriate OpenCV matcher"	Igual
`cv2.xfeatures2d.BriefDescriptorExtractor_create()` , `cv2.BRISK_create()` , `cv2.xfeatures2d.FREAK_create()`	`descriptors.py`	§2.5: BRIEF e FREAK podem exigir o `opencv-contrib-python`	Igual
Normas: `NORM_L2` (SIFT, KAZE), `NORM_HAMMING` (ORB, BRIEF, BRISK, FREAK)	`descriptors.py`	"For binary descriptors, use Hamming distance; for SIFT-like floating-point descriptors, use an L2-compatible metric"	Igual; coincide com `defaultNorm()` de cada objeto e com a recomendação da documentação do `BFMatcher`
Figuras de matches em Matplotlib	`visualization.py`	"Use cv2.drawMatches() or custom Matplotlib visualizations"	Igual (segunda opção)
`cv2.findHomography` , `cv2.warpPerspective`	—	Indicadas para a homografia e o warping	Parte da Pessoa B (por fazer)
`cv2.Stitcher`	—	"Students should avoid relying on cv2.Stitcher"	Não usado

Outras funções do OpenCV usadas: `detect` e `compute` (em chamadas separadas, para medir os dois tempos), `perspectiveTransform` , `GaussianBlur` , `cvtColor` , `imdecode` / `imencode` , `getRotationMatrix2D` , `warpAffine` , `resize` , `setRNGSeed` , `setNumThreads` , `FlannBasedMatcher` (opção).

2.2 Parâmetros: config vs valores por omissão do OpenCV 4.13.0

Objeto	Parâmetro	Omissão OpenCV	Config	Estado
SIFT	nOctaveLayers / contrastThreshold / edgeThreshold / sigma	3 / 0,04 / 10 / 1,6	iguais	Igual
SIFT	nfeatures	0 (todos)	0, com corte nosso aos 2000	Igual + limite comum (D05)
ORB	scaleFactor / nlevels / edgeThreshold / fastThreshold	1,2 / 8 / 31 / 20	iguais	Igual
ORB	nfeatures	500	2000	Diferente, de propósito: limite comum (D10)
ORB	WTA_K / patchSize / scoreType	2 / 31 / Harris	não definidos (omissão)	Igual; com WTA_K = 2 a norma certa é `NORM_HAMMING`
KAZE	extended / upright / threshold	False / False / 0,001	iguais	Igual
KAZE	nOctaves / nOctaveLayers / diffusivity	4 / 4 / PM_G2	não definidos (omissão)	Igual
FAST	**threshold**	**10**	**20**	**Diferente** (ver 2.3)
FAST	nonmaxSuppression / type	True / TYPE_9_16	iguais	Igual
BRIEF	bytes / use_orientation	32 / False	iguais	Igual
BRISK	thresh / octaves / patternScale	30 / 3 / 1,0	iguais	Igual
FREAK	orientationNormalized / scaleNormalized / patternScale / nOctaves	True / True / 22,0 / 4	iguais	Igual

Dimensão dos descritores confirmada na versão instalada: SIFT 128 floats; KAZE 64 floats; ORB 32 bytes; BRIEF 32 bytes; BRISK 64 bytes; FREAK 64 bytes.

2.3 Erro encontrado: o threshold do FAST

- A entrada D09 do DECISOES.md dizia que todos os parâmetros dos detetores eram "os valores por omissão do OpenCV". **No FAST isso estava errado:** a omissão é 10 e nós usamos 20.
- **Origem do 20:** o código de arranque do guia da cadeira (`docs/starter.py` , `FastFeatureDetector_create(threshold=20, ...)`). 20 é também o `fastThreshold` por omissão do ORB.
- **Impacto:** todas as configurações FAST+BRIEF/BRISK/FREAK das experiências principais usam t = 20. Com o limite de 2000 pontos (os de maior resposta), o efeito de usar 10 ou 20 só existe nas imagens em que o FAST com t = 20 encontra menos de 2000 pontos. O estudo do FAST (D24) mostra que t = 20 com NMS dá em média 5,7 mil (`i`) e 9,5 mil (`v`) pontos por imagem.
- **Decisão pendente:** manter 20 (documentado como escolha) ou passar a 10 e repetir as corridas. A D09 foi corrigida para não afirmar o que não é verdade.

2.4 Bibliotecas

Biblioteca	Recomendada no enunciado (§2.1)	Usada	Para quê
OpenCV 4.13.0.92 (contrib)	Sim	Sim	Detetores, descritores, matching, geometria
NumPy	Sim	Sim	Cálculo vetorial, BRIEF próprio
SciPy	Sim	Sim	KD-tree e emparelhamento bipartido máximo (correspondências ground-truth)
Matplotlib	Sim	Sim	Figuras
pandas	Sim (“for storing experimental results”)	Sim	Tabelas e CSV
scikit-image	Sim (“where useful”)	Não	Não foi necessária
psutil	Sim (“optional memory measurements”)	Não	A memória dos descritores é calculada diretamente (`desc.nbytes`). Está no `requirements.txt` sem ser usada
PyTorch	Sim (métodos aprendidos)	Ainda não	Parte da Pessoa B

Nenhuma biblioteca fora da lista do enunciado é usada.

2.5 Códigos de exemplo dos professores

Os três notebooks (`códigos exemplo 2026/`) **não foram a base do código**. O código foi escrito a partir do enunciado, que é a fonte que define o protocolo (3 px, ratio 0,8, emparelhamento 1-para-1, métricas obrigatórias). Segundo o guia da cadeira, os notebooks usam escolhas diferentes das do enunciado nalguns pontos (por exemplo, 5 px e só cross-check no notebook do SIFT). No relatório não se deve afirmar que partimos desses códigos.